PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

Строим очередь и кэш на Postgres без Redis, RabbitMQ и Kafka

Стас Выщепан

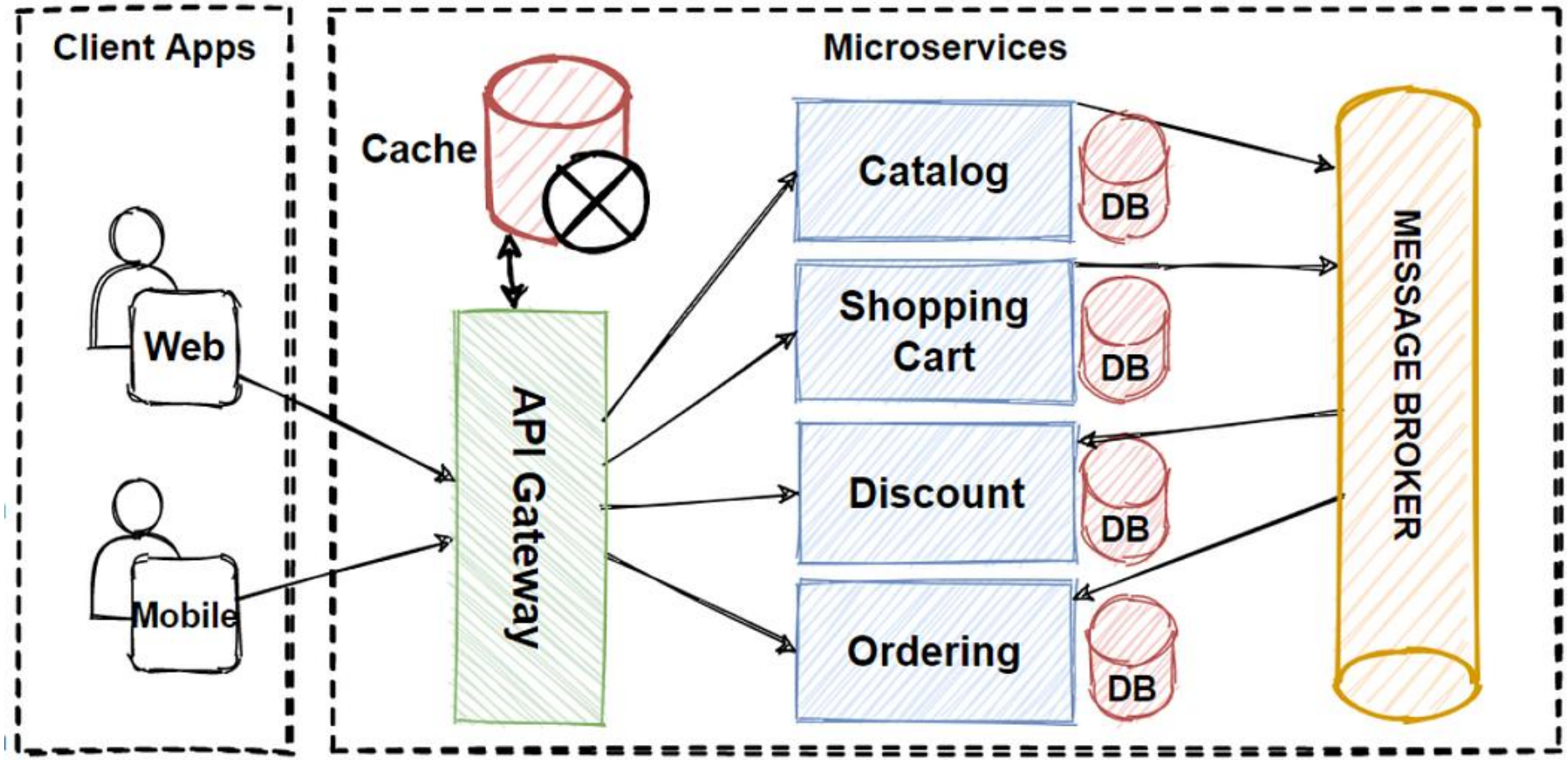
Руководитель разработки продукта, Миртех

me@stanislav.ru

Обо мне

- Создаю программы с 2007 года
- Microsoft MVP 2011-2018 (SharePoint, SQL Server, Windows Server)
- С 2022 занимаюсь Postgres и Linux
- Сейчас руковожу командой разработки продукта
  - Мультитенантное веб-приложение, раздает 100к+ страниц в день
  - 30-50 запросов в секунду на бэкенд в рабочее время
  - Не хайлоад
  - Микросервисная архитектура
  - SQL Server, плавно мигрирующий в Postgres

Современная архитектура



Рандомная картинка из интернета

Современная архитектура на практике

- Кэш – Redis или Valkey
- Очереди сообщений – Kafka или RabbitMQ
- Даже в случае монолитной архитектуры кэши и очереди используются для горизонтального масштабирования

- Базы данных – в подавляющем большинстве Postgres
  - На земле - отдельный сервер \ кластер
  - В облаке - managed сервис
- Остальное в контейнерах, конкурирует за ресурсы хоста

Дополнительная стоимость архитектуры

- *Eventual consistency* – эвфемизм для несогласованных данных
  - Нужно писать дополнительный код для обработки несогласованности
  - Повышенная вероятность потери данных и недовольство клиента
- Больше точек отказа
  - Больше инфраструктуры для отказоустойчивости
  - Больше специалистов для подержания работоспособности
- Медленное взаимодействие
  - «Давайте добавим кэш»
  - «Давайте ресурсов»
  - «Давайте добавим еще инстанс кафки»

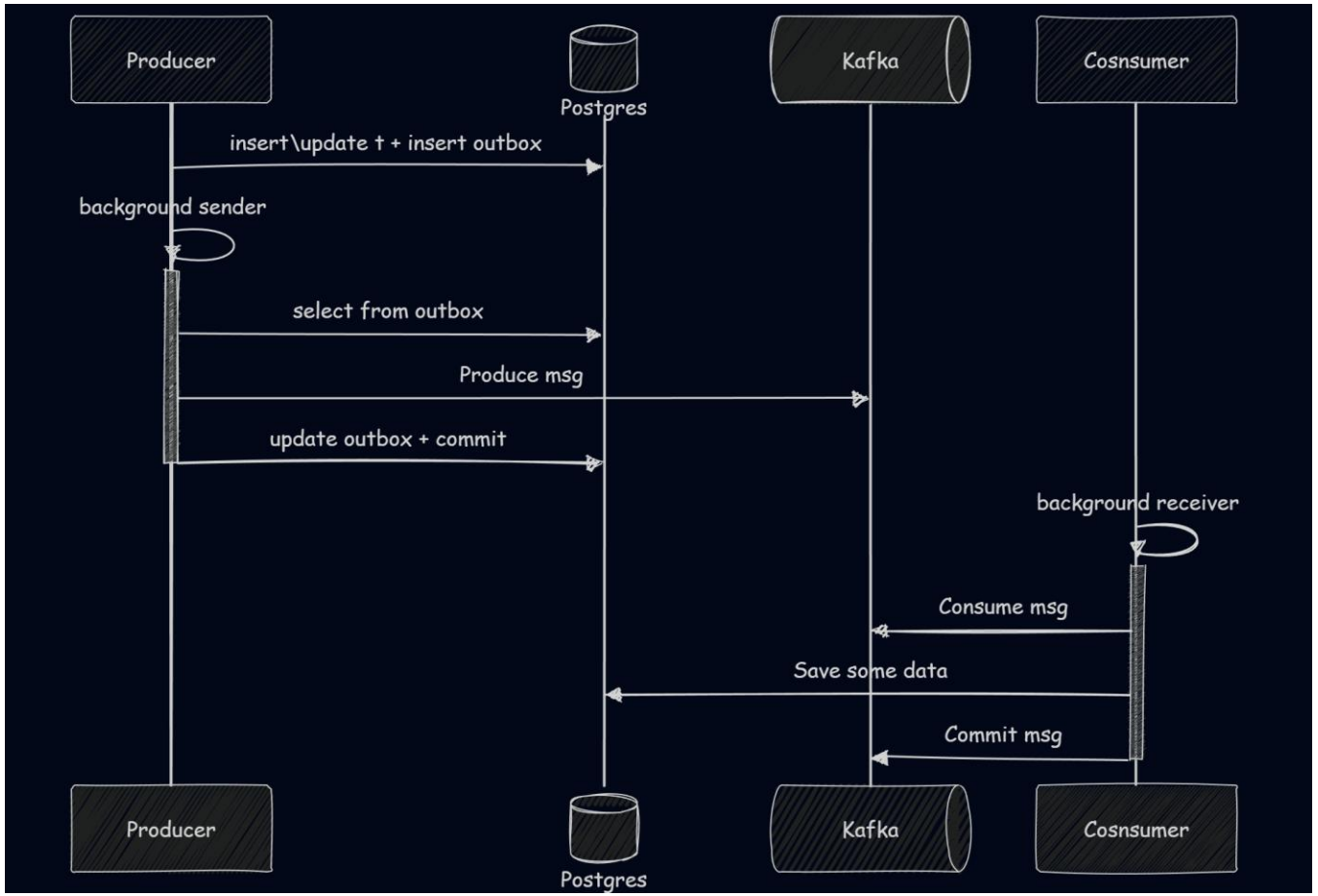
Apache Kafka и RabbitMQ Streams

- «Append-only log of messages»
- Pull – модель, консьюмер сам обращается за новыми сообщениями
- Сообщения доставляются строго в порядке появления
- Консьюмер независимые, каждый сам обновляет метку последнего сообщения
- Хранится вся история, можно «перемотать»
- Надежные, реплицируемые очереди сообщений
- «Dumb queue, smart consumer»

Очереди надежные, а отправка - нет

- Типичный сценарий для микросервисов:
  1. Сервис получает данные извне: веб-запрос или читает из очереди
  2. Делает работу
  3. Сохраняет результат в базе
  4. Отправляет в очередь сообщение
- Что будет, если произойдет отказ между 3 и 4?
- Решение – transactional outbox
  - У каждого сервиса своя локальная очередь в базе
  - В одной транзакции пишется и результат работы, и сообщение для отправки
  - Фоновая задача отправляет сообщения и повторяет в случае проблем

Kafka + Transactional Outbox



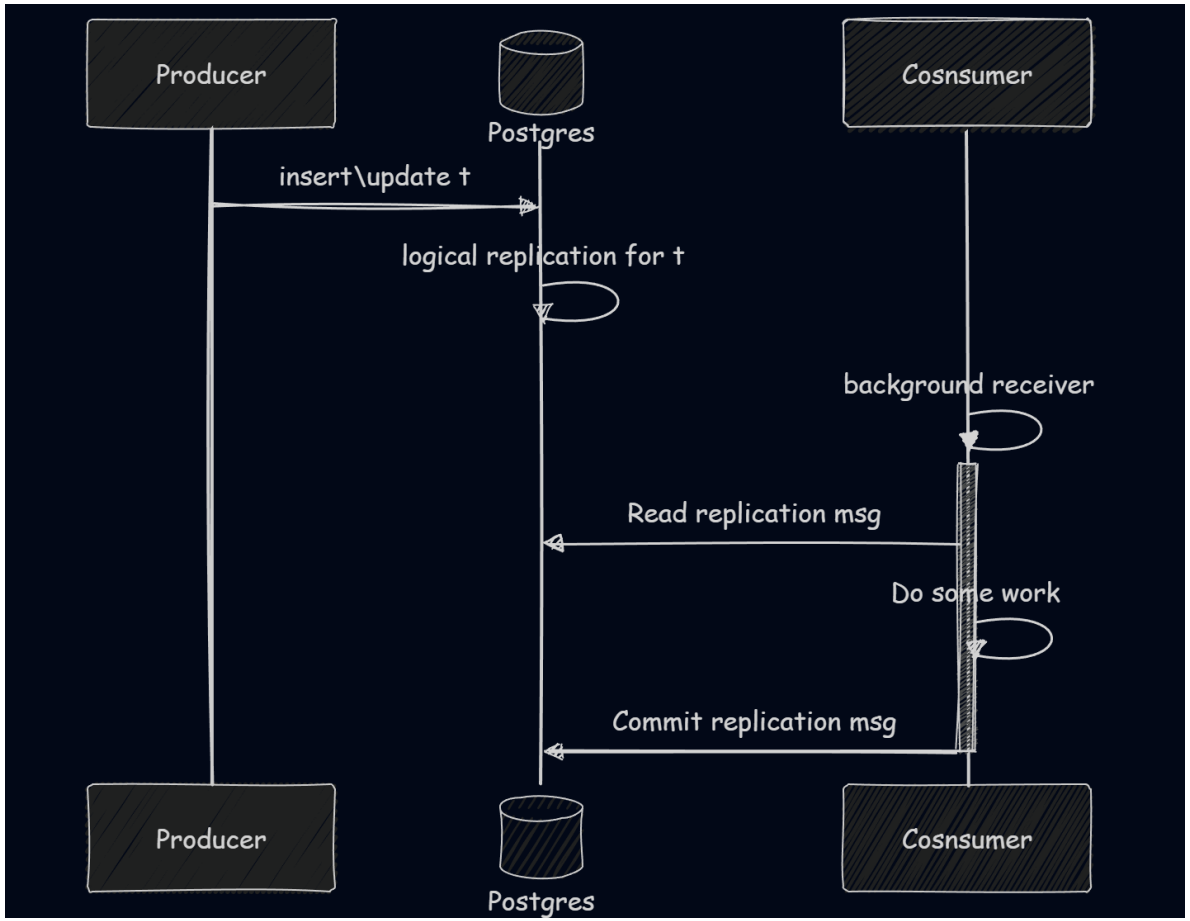
- Надежно, но…
- Write amplification
  - Insert + update на каждое сообщение
  - Vacuum приходит чаще
- Все равно встречаются на Postgres

Логическая репликация в Postgres

- Подписка читает WAL и формирует очередь событий с параметрами публикации
- Подписчик публикации (slot) запрашивает новые события и меняет метку последнего прочитанного
- События считываются строго последовательно

- Вам тоже это что-то напоминает?

Postgres вместо Kafka



- Меньше точек отказа
- Нет write amplification
- Меньше записей в сумме

Проверяем быстродействие

Логическая репликация Postgres vs Apache Kafka

Тестовый стенд

- Локальный компьютер
  - 16 ядер
  - 64ГБ памяти
  - SSD M2 диск
- Все сервисы запускаются в docker
- Тестовые приложения на компьютере
- Нагрузка с помощью k6

Postgres вместо Kafka

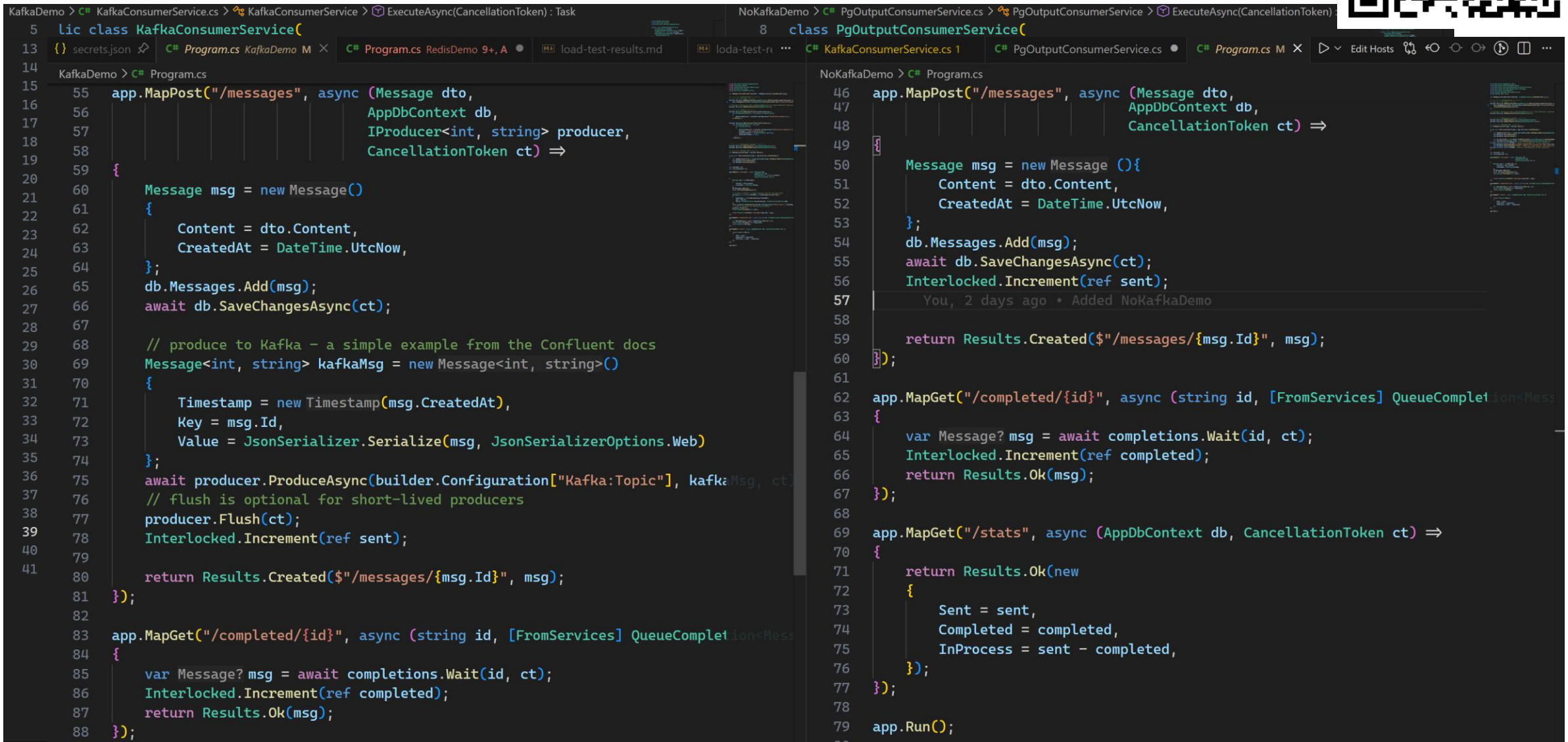
KafkaDemo > KafkaConsumerService.cs > KafkaConsumerService > ExecuteAsync(CancellationToken) : Task

```
5   lic class KafkaConsumerService(
13   protected override async Task ExecuteAsync(CancellationToken stoppingToken
14   {
15       consumer.Subscribe(config["Kafka:Topic"]);
16       while (!stoppingToken.IsCancellationRequested)
17       {
18           try
19           {
20               var ConsumeResult<int, string>? consumed = consumer.Consume(st
21
22               try
23               {
24                   logger.LogInformation("Consumed: Key={Key}, Value={Value}",
25                   completions.Complete(consumed.Message.Key.ToString(), Jsonl
26               }
27               finally
28               {
29                   consumer.Commit(consumed);
30                   logger.LogInformation("Commited: Key={Key}", consumed.Mess
31               }
32           }
33           catch (ConsumeException ex)
34           {
35               logger.LogError(ex, "Consume error");
36           }
37       }
38       consumer.Close();
39   }
40
41
```

NoKafkaDemo > PgOutputConsumerService.cs > PgOutputConsumerService > ExecuteAsync(CancellationToken)

```
8   class PgOutputConsumerService(
17  tected override async Task ExecuteAsync(CancellationToken stoppingToken)
19   await using var AppDbContext? ctx = await dbFactory.CreateDbContextAsync(s
20   var IModel? model = ctx.Model;
21   var IRelationalModel? relModel = ctx.Model.GetRelationalModel();
22  ou, 2 days ago • Added NoKafkaDemo
23   await using var LogicalReplicationConnection? conn = new LogicalReplicatio
24   await conn.Open(stoppingToken);
25
26   var PgOutputReplicationSlot? slot = new PgOutputReplicationSlot("rep_slot");
27
28   // The following will loop until the cancellation token is triggered, and
29   await foreach (var PgOutputReplicationMessage message in conn.StartReplica
30       slot,
31       new PgOutputReplicationOptions("rep_pub", PgOutputProtocolVersion.V4,
32       stoppingToken))
33   {
34       try
35       {
36
37           if (message is InsertMessage insertMessage)
38           {
39               logger.LogInformation("Received new row: {Namespace}.{Table} W
40                   insertMessage.Relation.Namespace,
41                   insertMessage.Relation.RelationName,
42                   insertMessage.WalStart,
43                   insertMessage.WalEnd
44                   );
45               Message msg = await DeserializeMessage(relModel, insertMessage,
46               completions.Complete(msg.Id.ToString(), msg);
47           }
48       }
49       finally
50       {
51           // Always call SetReplicationStatus() or assign LastAppliedLsn and
52           // so that Npgsql can inform the server which WAL files can be rem
53           conn.SetReplicationStatus(message.WalEnd);
54           await conn.SendStatusUpdate(stoppingToken);
55           logger.LogInformation("Replication flushed: WalStart:{WalStart} Wa
```

Postgres вместо Kafka



KafkaDemo > KafkaConsumerService.cs > KafkaConsumerService > ExecuteAsync(CancellationToken) : Task

```
lic class KafkaConsumerService(
```

secrets.json | Program.cs KafkaDemo M | Program.cs RedisDemo 9+, A | load-test-results.md | loda-test-re | KafkaConsumerService.cs 1 | PgOutputConsumerService.cs | Program.cs M | Edit Hosts

KafkaDemo > Program.cs

```
app.MapPost("/messages", async (Message dto,
                                AppDbContext db,
                                IProducer<int, string> producer,
                                CancellationToken ct) =>
{
    Message msg = new Message()
    {
        Content = dto.Content,
        CreatedAt = DateTime.UtcNow,
    };
    db.Messages.Add(msg);
    await db.SaveChangesAsync(ct);

    // produce to Kafka - a simple example from the Confluent docs
    Message<int, string> kafkaMsg = new Message<int, string>()
    {
        Timestamp = new Timestamp(msg.CreatedAt),
        Key = msg.Id,
        Value = JsonSerializer.Serialize(msg, JsonSerializerOptions.Web)
    };
    await producer.ProduceAsync(builder.Configuration["Kafka:Topic"], kafkaMsg, ct
    // flush is optional for short-lived producers
    producer.Flush(ct);
    Interlocked.Increment(ref sent);

    return Results.Created($"/messages/{msg.Id}", msg);
});

app.MapGet("/completed/{id}", async (string id, [FromServices] QueueComplet
{
    var Message? msg = await completions.Wait(id, ct);
    Interlocked.Increment(ref completed);
    return Results.Ok(msg);
});
```

NoKafkaDemo > PgOutputConsumerService.cs > PgOutputConsumerService > ExecuteAsync(CancellationToken)

```
class PgOutputConsumerService(
```

NoKafkaDemo > Program.cs

```
app.MapPost("/messages", async (Message dto,
                                AppDbContext db,
                                CancellationToken ct) =>
{
    Message msg = new Message (){
        Content = dto.Content,
        CreatedAt = DateTime.UtcNow,
    };
    db.Messages.Add(msg);
    await db.SaveChangesAsync(ct);
    Interlocked.Increment(ref sent);
      You, 2 days ago • Added NoKafkaDemo

    return Results.Created($"/messages/{msg.Id}", msg);
});

app.MapGet("/completed/{id}", async (string id, [FromServices] QueueComplet
{
    var Message? msg = await completions.Wait(id, ct);
    Interlocked.Increment(ref completed);
    return Results.Ok(msg);
});

app.MapGet("/stats", async (AppDbContext db, CancellationToken ct) =>
{
    return Results.Ok(new
    {
        Sent = sent,
        Completed = completed,
        InProcess = sent - completed,
    });
});

app.Run();
```

Сценарий Kafka vs Postgres

1. Отправить делать веб-запрос
2. Дождаться считывания сообщения и очереди

- 250 Virtual User (VUS)
- 100 000 запросов

Результат Kafka vs Postgres

Показатель	Kafka queue	Postgres queue
Время выполнения теста	01m19.1s	00m16.1s
Тестов в сек	1263.94441	6215.251848
CPU Postgres до теста	0.03%	0.00%
MEM Postgres до теста	61.63MiB	49.63MiB
CPU Kafka до теста	2.51%	1.07%
MEM Kafka до теста	326.2MiB	912.7MiB
CPU Postgres во время теста	35.58%	323.95%
MEM Postgres во время теста	242.3MiB	235.2MiB
CPU Kafka во время теста	68.43%	1.81%
MEM Kafka во время теста	926.4MiB	912MiB

Postgres вместо Kafka

- Topic \ Stream → Таблица
- Partition → PUBLICATION with filter
- Consumer → Replication slot
- Физический WAL преобразуется в сообщения логической репликации
  - Штатный плагин pgoutput – бинарный протокол
  - Штатный плагин test_decoding – текстовый, для изучения
  - Сторонний wal2json – если провайдер на вашем языке не поддерживает pgoutput

- Можно подключаться даже к реплике

Можно совсем отказаться от Kafka?

- Если вы создаете и отправителя, и получателя – да

- Kafka умеет распределять нагрузку автоматически
- Может быть оправдано при масштабировании для очень больших нагрузок на передачу сообщений

- Для Postgres надо учитывать необходимое количество ядер и лицензирование
- Репликация требует мониторинга и обслуживания

Различия RabbitMQ и Kafka

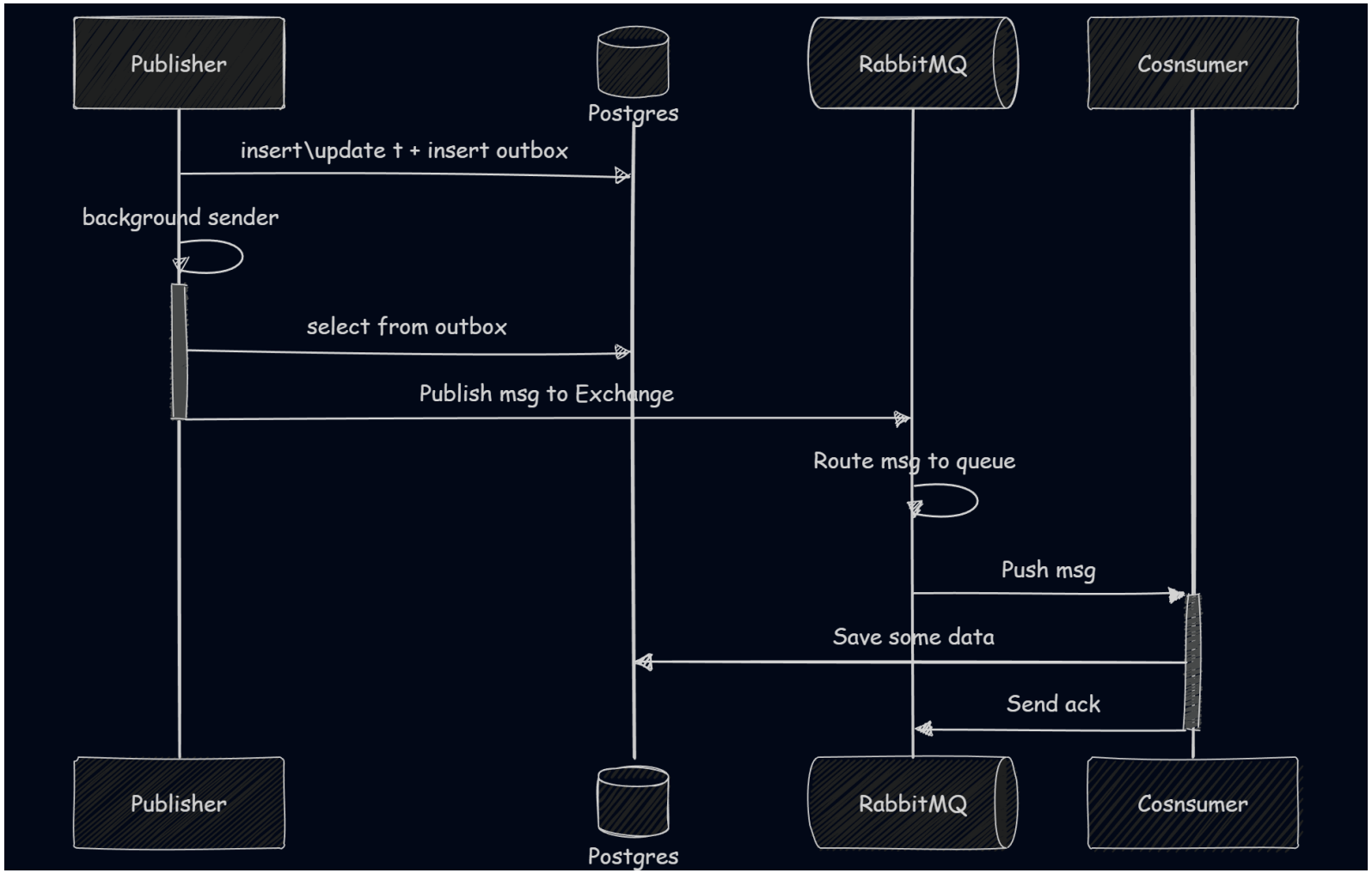
- RabbitMQ по умолчанию очень ненадежен
- «Smart queue» vs «Dumb queue»
- Push-модель – RabbitMQ рассылает сообщения подписчикам
- RabbitMQ умеет маршрутизировать сообщения
- Есть обработка poison message и dead letter queue
- Имеет плагины для веб-сокетов для работы с разными протоколами, включая http и ws
- Поддерживает несколько читателей для одной очереди – может распределять нагрузку

Надежные очереди RabbitMQ

- Quorum queue
  - Надежные очереди, но требуют кластера
  - Используют репликацию с кворумом
  - Несколько читателей для одной очереди
  - Гарантия доставки at-least-once даже при падении рэббита

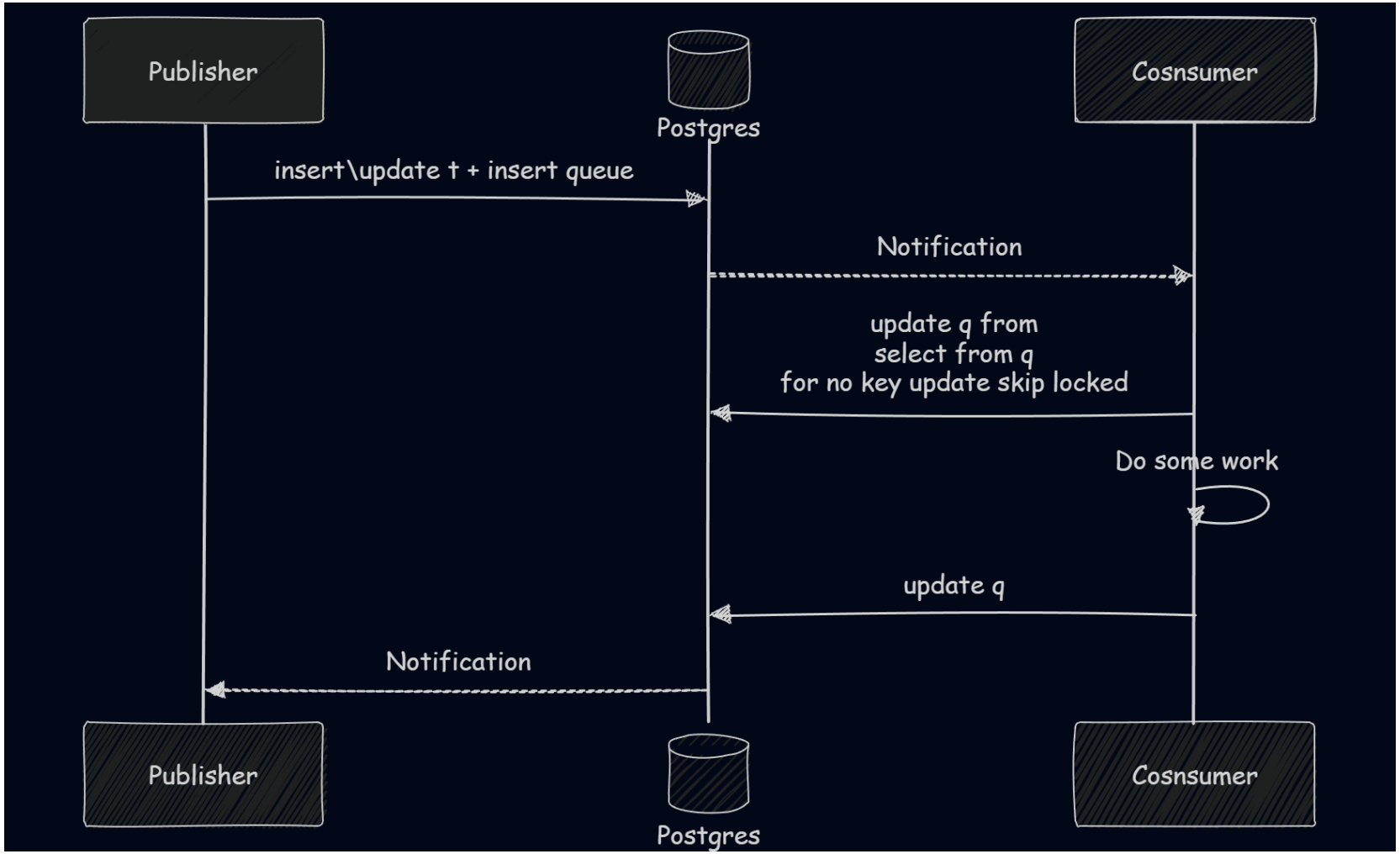
- Durable queue
  - Работают на одном сервере
  - Могут потеряться при отказе сервера сразу после отправки
  - Несколько читателей для одной очереди
  - Могут доставляться несколько раз в кластере

Transactional outbox и очереди RabbitMQ



- Сообщения всегда попадают в очередь
- Один запрос обрабатывается максимум один раз
- Параллельность ограничена количеством исполнителей

Можно ли без RabbitMQ ?



- Write Amplification
- Можно сделать poison message handling
- Можно иметь несколько обработчиков

Оповещения в Postgres

- Команда LISTEN {channel}
  - Включает прослушивание оповещений указанного канала в текущем соединении

- Команда NOTIFY и функция pg_notify для отправки оповещений
  - Отправляет оповещение всем слушателям канала при коммите транзакции
  - Можно использовать в триггерах

UNLOGGED таблицы

- Очищаются при аварийном завершении СУБД
- Быстрее обычных так как не пишут WAL
- Не участвуют в репликации
- Только мастер
- Участвуют в транзакциях
- Используют shared buffers

Проверяем быстродействие

Сравнение RabbitMQ и Postgres

Сценарий RabbitMQ vs Postgres

1. Отправить делать веб-запрос
2. Дождаться считывания сообщения и очереди

- 250 Virtual User (VUS)
- 100 000 запросов
- Один инстанс Rabbit с обычными durable очередями
- Сравниваем с решениями на PG:
  - WAL queue (как для Kafka)
  - UPDATE FROM SELECT SKIP LOCKED + UNLOGGED таблицы

Результат RabbitMQ vs Postgres

Показатель	RabbitMQ	Postgres WAL	Postgres UNLOGGED
Время выполнения теста	02m21.6s	00m16.1s	04m43.7s
Тестов в сек	706.015423	6215.251848	352.497123
CPU Postgres до теста	0.01%	0.00%	0.00%
MEM Postgres до теста	224MiB	49.63MiB	164.1MiB
CPU RabbitMQ до теста	0.64%	-	
MEM RabbitMQ до теста	93.77MiB	-	
CPU Postgres во время теста	26.62%	323.95%	137.17%
MEM Postgres во время теста	318.2MiB	235.2MiB	345.9MiB
CPU RabbitMQ во время теста	85.18%	-	
MEM RabbitMQ во время теста	118.1MiB	-	

Код RabbitMQ vs Postgres

RabbitMqDemo > C# Program.cs

```
34  app.MapPost("/messages", async (Message dto,
35                                  AppDbContext db,
36                                  IBus bus,
37                                  CancellationToken ct) =>
38  {
39      Message msg = new Message()
40      {
41          Content = dto.Content,
42          CreatedAt = DateTime.UtcNow,
43      };
44      db.Messages.Add(msg);
45      await db.SaveChangesAsync(ct);
46
47      await bus.SendReceive.SendAsync(nameof(Message), msg,
48      ISendConfiguration c => c.WithPublisherConfirms(true), ct);
49      Interlocked.Increment(ref sent);
50
51      return Results.Created($"/messages/{msg.Id}", msg);
52  });
53
54  app.MapGet("/completed/{id}", async (string id, [FromServices] QueueCompletion<Message> complet
55  {
56      var Message? msg = await completions.Wait(id, ct);
57      Interlocked.Increment(ref completed);
58      return Results.Ok(msg);
59  });
60
61  app.MapGet("/stats", async (AppDbContext db, CancellationToken ct) =>
62  {
63      return Results.Ok(new
64      {
65          Sent = sent,
66          Completed = completed,
67          InProcess = sent - completed,
68      });
69  });
70
71  app.Run();
72
```

NoRabbitMqDemo > C# Program.cs

```
31  app.MapPost("/messages", async (Message dto,
32                                  AppDbContext db,
33                                  CancellationToken ct) =>
34  {
35      await using var IDbContextTransaction? t = await db.Database.BeginTransactionAsync(ct);
36      Message msg = new Message()
37      {
38          Content = dto
39          CreatedAt = Da
40      };
41      db.Messages.Add(ms
42      await db.SaveChang
43
44      db.Queue.Add(new Q
45      {
46          Request = new QueuePayload () { Content = JsonSerializer.SerializeToDocument(msg) }
47      });
48      await db.SaveChangesAsync(ct);
49      await db.Database.ExecuteSqlRawAsync("NOTIFY queue", ct);
50      await t.CommitAsync(ct);
51
52      Interlocked.Increment(ref sent);
53
54      return Results.Created($"/messages/{msg.Id}", msg);
55  });
56
57  app.MapGet("/completed/{id}", async (string id, [FromServices] QueueCompletion<Message> complet
58  {
59      var Message? msg = await completions.Wait(id, ct);
60      Interlocked.Increment(ref completed);
61      return Results.Ok(msg);
62  });
63
64  app.MapGet("/stats", async (AppDbContext db, CancellationToken ct) =>
65  {
66      return Results.Ok(new
67      {
68          Sent = sent,
69          Completed = completed,
```

```
JsonDocument JsonSerializer.SerializeToDocument<Message>(Message value,
[JsonSerializerOptions? options = null]) (+ 4 overloads)

Converts the provided value into a JsonDocument.

Returns:
  A JsonDocument representation of the JSON value.

Exceptions:
  NotSupportedException
```

Код RabbitMQ vs Postgres

RabbitMqDemo > C# Program.cs

```
app.MapPost("/messages", async (Message dto,
```

RabbitMqDemo > C# RabbitConsumerService.cs > ...

You, 3 hours ago | 1 author (You)

```
using EasyNetQ;

1 reference | You, 3 hours ago | 1 author (You)
public class RabbitMqConsumerService(
    IBus bus,
    QueueCompletion<Message> completions)
    : IHostedService
{
    2 references
    IAsyncDisposable subscription = null!;
    0 references
    public async Task StartAsync(CancellationToken cancellationToken)
    {
        subscription = await bus.SendReceive.ReceiveAsync<Message>(nameof(@Message), Message m =>
        {
            completions.Complete(m.Id.ToString(), m);
        }, IReceiveConfiguration c=> c.WithDurable(), cancellationToken);
    }

    0 references
    public async Task StopAsync(CancellationToken cancellationToken)
    {
        await subscription.DisposeAsync();
    }
}
```

NoRabbitMqDemo > C# Program.cs

```
app.MapPost("/messages", async (Message dto,
```

NoRabbitMqDemo > C# PgNotificationConsumerService.cs > ...

```
:lass PgNotificationConsumerService(
vate async Task Run(CancellationToken stoppingToken)
while (!stoppingToken.IsCancellationRequested)
{
    var List<QueueItem>? q = await ctx.Database.SqlQuery<QueueItem>($"""
        with processing as (
            update "queue"
            set
                started_at = {DateTime.UtcNow}
            from (
                select ctid
                from "queue"
                where processed_at is null
                    and (started_at is null or started_at < {DateTime.UtcNow.AddMinutes(-1)
                for no key update skip locked
            ) to_process
            where "queue".ctid = to_process.ctid
            returning "queue".id,"queue".created_at,"queue".request_payload_id
        )
        select q.id, p.content
        from processing q
        join queue_payloads p on p.id = q.request_payload_id
        order by q.created_at
    """).ToListAsync(stoppingToken);

    if (q.Count > 0 )
    {

        foreach (var QueueItem e in q)
        {
            var Message? msg = JsonSerializer.Deserialize<Message>(e.Content)!;
            completions.Complete(msg.Id.ToString(), msg);
        }
        var IEnumerable<Guid>? ids = q.Select(QueueItem e => e.Id);
        await ctx.Queue.Where(QueueItem e =>  ids.Contains(e.Id)).ExecuteUpdateAsync(
            UpdateSettersBuilder<QueueItem> p => p.SetProperty(QueueItem e => e.ProcessedAt,
            stoppingToken
        );
```

Код RabbitMQ vs Postgres

RabbitMqDemo > Program.cs

```
app.MapPost("/messages", async (Message dto,
```

RabbitMqDemo > RabbitConsumerService.cs > ...

You, 3 hours ago | 1 author (You)

```
using EasyNetQ;

1 reference | You, 3 hours ago | 1 author (You)
public class RabbitMqConsumerService(
    IBus bus,
    QueueCompletion<Message> completions)
    : IHostedService
{
    2 references
    IAsyncDisposable subscription = null!;
    0 references
    public async Task StartAsync(CancellationToken cancellationToken)
    {
        subscription = await bus.SendReceive.ReceiveAsync<Message>(nameof(@Message), Message m =>
        {
            completions.Complete(m.Id.ToString(), m);
        }, IReceiveConfiguration c => c.WithDurable(), cancellationToken);
    }

    0 references
    public async Task StopAsync(CancellationToken cancellationToken)
    {
        await subscription.DisposeAsync();
    }
}
```

NoRabbitMqDemo > Program.cs

```
app.MapPost("/messages", async (Message dto,
```

NoRabbitMqDemo > PgNotificationConsumerService.cs > ...

```
:lass PgNotificationConsumerService(
vate async Task Run(CancellationToken stoppingToken)
while (!stoppingToken.IsCancellationRequested)
{
    var List<QueueItem>? q = await ctx.Database.SqlQuery<QueueItem>($"""
        with processing as (
            update "queue"
            set
                started_at = {DateTime.UtcNow}
            from (
                select ctid
                from "queue"
                where processed_at is null
                    and (started_at is null or started_at < {DateTime.UtcNow.AddMinutes(-1)
                for no key update skip locked
            ) to_process
            where "queue".ctid = to_process.ctid
            returning "queue".id,"queue".created_at,"queue".request_payload_id
        )
```

```
var NpgsqlConnection? conn = (Npgsql.NpgsqlConnection)ctx.Database.GetDbConnection();
if (conn.State ≠ System.Data.ConnectionState.Open) await conn.OpenAsync(stoppingToken);
using (var NpgsqlCommand? cmd = conn.CreateCommand())
{
    {
        cmd.CommandText = "LISTEN queue";
        await cmd.ExecuteNonQueryAsync(stoppingToken);
    }
}
await conn.WaitAsync(stoppingToken);
```

```
ent)!;
            completions.Complete(msg.Id.ToString(), msg);
        }
        var IEnumerable<Guid>? ids = q.Select(QueueItem e => e.Id);
        await ctx.Queue.Where(QueueItem e => ids.Contains(e.Id)).ExecuteUpdateAsync(
            UpdateSettersBuilder<QueueItem> p => p.SetProperty(QueueItem e => e.ProcessedAt,
            stoppingToken
        );
```

Ограничения очередей в Postgres

- Сложная логика обработки очередей
  - Маршрутизация сообщений
  - Poison message handling
  - Dead letter exchange

- Масштабируемость
  - Запись только в мастер
  - Количество соединений
  - Повышенная нагрузка на диск

Сценарии использования Redis \ Valkey

1. GET\PUT\DEL по ключу
   - Хэш-таблица в памяти

2. Pub\Sub
   - Рассылка уведомлений

3. Остальное
   - Stream
   - Bitmap
   - Vector
   - JSON inverted index

Почему кэш помогает

- Хэш-индекс в памяти быстрее btree
- Снижает количество подключений к БД
- Нет очередей в пуле под нагрузкой

- Запрос строки по ключу с кэшированием в Redis:

Сценарий	Запросов\сек	CPU Postgres	CPU Redis	MEM Postgres	MEM Redis
Без кэша	9795.251129	480.98%	0.21%	1.186GiB	32.66MiB
Cache hit 1%	9219.264145	398.06%	19.01%	1.187GiB	318.9MiB
Cache hit 10%	10522.616602	374.69%	27.48%	1.188GiB	294.4MiB
Cache hit 50%	13723.633997	276.73%	24.13%	1.186GiB	180.6MiB
Cache hit 90%	27483.044796	79.91%	19.70%	1.185GiB	63.09MiB

Redis \ Valkey тоже можно заменить?

- Для простых запросов строк по Id
  - На кэш в памяти приложения
  - Сбрасывать с помощью логической репликации или оповещений в Postgres

- Для сложных запросов
  - REFRESH MATERIALIZED VIEW
  - UNLOGGED таблицы в Postgres с хэш-индексами
  - Сброс с помощью триггеров

Проверяем быстродействие

Кэш в Redis vs в Postgres + приложении

Сценарии

Запрос по ключу одной строки	«Тяжелый» запрос + обновление со сбросом кэша	Чтение и запись кэша в один поток за минуту

• 200 Virtual User (VUS)
• 1 000 000 запросов
• Целевой cache hit = 90%
	• 1 минута
	• Случайный ключ из 10 ключей
	• «Тяжелый» запрос
	  • 100 VUS
	• Обновление
	  • 1 VUS
	• Целевой cache hit = 99%
		• 1 минута
		• 50кб JSON (jsonb колонка в Postgres)

Результаты

Сценарий	Запросов\сек PG	Запросов\сек Redis
Запрос по ключу	35 073.277353	12 919.073876
Тяжелый запрос	25.447551	21.343674
Чтение-запись	113.800091	137.268136

Postgres вместо Redis

RedisDemo > Program.cs

```
128 app.MapPost("/transactions", async (Transaction t,
129 [FromServices] AppDbContext ctx,
130 [FromServices] IDistributedCache cache,
131 CancellationToken ct
132 ) =>
133 {
134     t.Id = 0;
135     if (!t.Credit.EndsWith('.')) t.Credit += '.';
136     if (!t.Debit.EndsWith('.')) t.Debit += '.';
137     ctx.Transactions.Add(t);
138     await ctx.SaveChangesAsync(ct);
139     await cache.RemoveAsync("credit:" + t.Credit, ct);
140     await cache.RemoveAsync("debit:" + t.Debit, ct);
141
142     return Results.Created($"/transactions/{t.Id}", t);
143 })
144 .WithName("AddTransaction");
145         You, yesterday • Added RedisDemo
146
147 app.MapGet("/transactions/{id}", async (long id,
148 [FromServices] AppDbContext ctx,
149 [FromServices] IDistributedCache cache,
150 [FromQuery] bool? nocache,
151 CancellationToken ct
152 ) =>
153 {
154     Transaction? tx;
155     if (nocache is true)
156     {
157         tx = await ctx.Transactions.FindAsync(id, ct);
158     }
159     else
160     {
161         var string? cacheKey = $"tx:{id}";
162         var string? cacheValue = await cache.GetStringAsync(cacheKey, ct);
163         if (cacheValue is { } string? v)
164         {
165             tx = JsonSerializer.Deserialize<Transaction>(v, JsonSerializerOptions.Web);
166         }
167         else
168         {
169             tx = await ctx.Transactions.FindAsync(id, ct);
170             if (tx is not null)
171                 await cache.SetStringAsync(cacheKey, JsonSerializer.Serialize(tx, JsonSerialize
172         }
173     }
174     return tx is { } Transaction? t ? Results.Ok(t) : Results.NotFound();
```

NoRedisDemo > Program.cs

```
95  app.MapPost("/transactions", async (Transaction t,
96  [FromServices] AppDbContext ctx,
97  [FromServices] IMemoryCache cache,
98  CancellationToken ct
99  ) =>
100 {
101     t.Id = 0;
102     if (!t.Credit.EndsWith('.')) t.Credit += '.';
103     if (!t.Debit.EndsWith('.')) t.Debit += '.';
104     ctx.Transactions.Add(t);
105     await ctx.SaveChangesAsync(ct);
106
107     return Results.Created($"/transactions/{t.Id}", t);
108 })
109 .WithName("AddTransaction");
110
111
112 app.MapGet("/transactions/{id}", async (long id,
113 [FromServices] AppDbContext ctx,
114 [FromServices] IMemoryCache cache,
115 [FromQuery] bool? nocache,
116 CancellationToken ct
117 ) =>
118 {
119     Transaction? tx;
120     if (nocache is true)
121     {
122         tx = await ctx.Transactions.FindAsync(id, ct);
123     }
124     else
125     {
126         var string? cacheKey = $"tx:{id}";
127         tx = await cache.GetOrCreateAsync(cacheKey, async ICacheEntry e =>
128         {
129             e.SetSlidingExpiration(TimeSpan.FromMinutes(1));
130             return await ctx.Transactions.FindAsync(id, ct);
131         });
132     }
133     return tx is { } Transaction? t ? Results.Ok(t) : Results.NotFound();
134 })
135 .WithName("GetTransaction");
136
137 app.MapPost("/prepare-cache", async (
138 [FromServices] AppDbContext ctx,
139 CancellationToken ct
140 ) =>
141 {
```

Postgres вместо Redis

RedisDemo > C# Program.cs

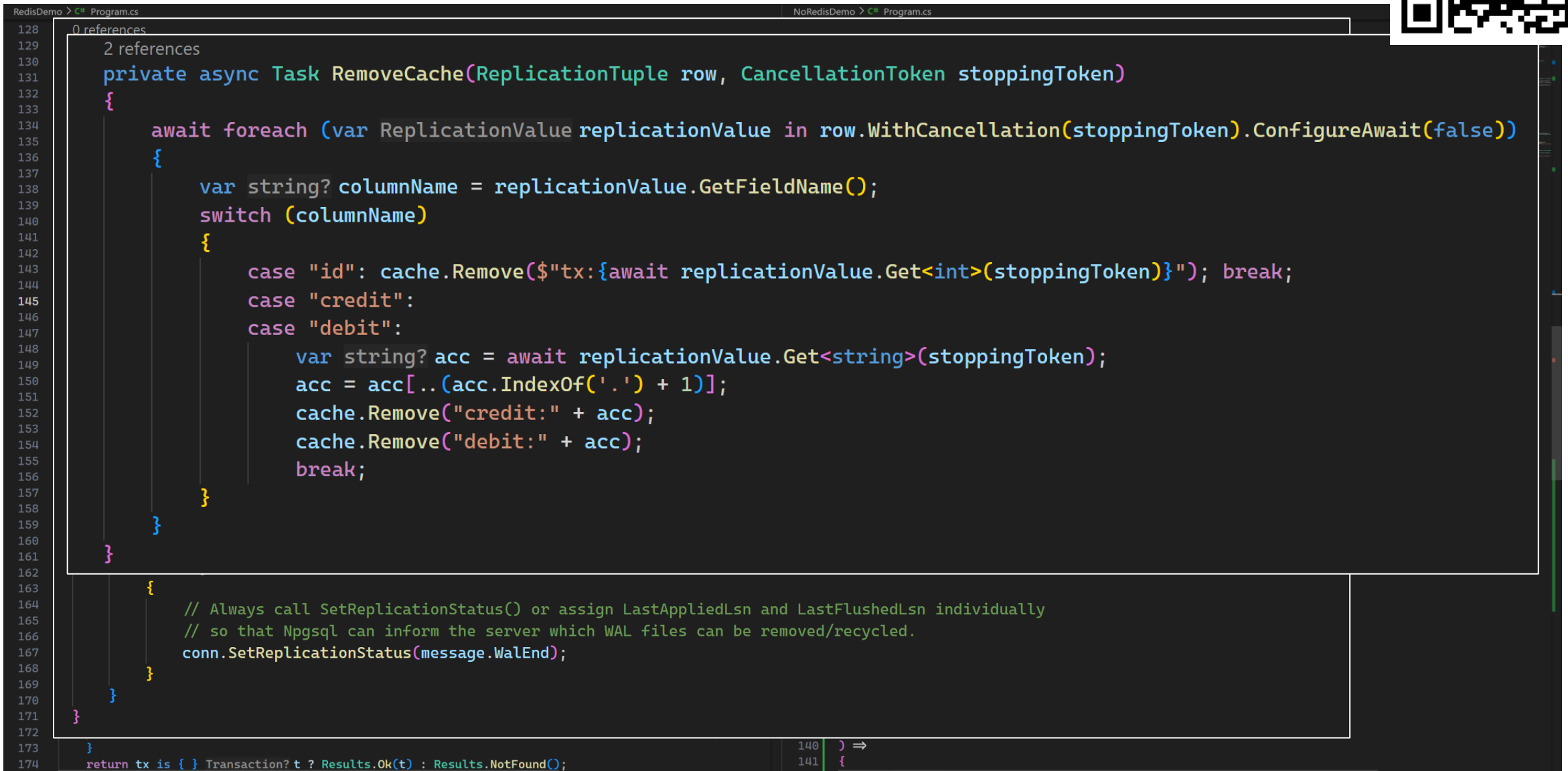
```
0 references
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    await using var LogicalReplicationConnection? conn = new LogicalReplicationConnection(config.GetConnectionString("PostgresRedisDemo"));
    await conn.Open(stoppingToken);

    // The following will loop until the cancellation token is triggered, and will print message types coming from PostgreSQL:
    await foreach (var PgOutputReplicationMessage message in conn.StartReplication(
        await conn.CreatePgOutputReplicationSlot("tx_slot", true, cancellationToken: stoppingToken),
        new PgOutputReplicationOptions("tx_pub", PgOutputProtocolVersion.V4, binary: true),
        stoppingToken))
    {
        try
        {
            switch (message)
            {
                case UpdateMessage m when m.Relation.RelationName == "transactions":
                    await RemoveCache(m.NewRow, stoppingToken).ConfigureAwait(false);
                    break;
                case InsertMessage m when m.Relation.RelationName == "transactions":
                    await RemoveCache(m.NewRow, stoppingToken).ConfigureAwait(false);
                    break;
                // Delete need to handle with notifications
            }
        }
        finally
        {
            // Always call SetReplicationStatus() or assign LastAppliedLsn and LastFlushedLsn individually
            // so that Npgsql can inform the server which WAL files can be removed/recycled.
            conn.SetReplicationStatus(message.WalEnd);
        }
    }
}
```

NoRedisDemo > C# Program.cs

```
    }
    return tx is { } Transaction? t ? Results.Ok(t) : Results.NotFound();
```

```
) =>
{
```

Postgres вместо Redis



RedisDemo > C# Program.cs

```
0 references
2 references
private async Task RemoveCache(ReplicationTuple row, CancellationToken stoppingToken)
{
    await foreach (var ReplicationValue replicationValue in row.WithCancellation(stoppingToken).ConfigureAwait(false))
    {
        var string? columnName = replicationValue.GetFieldName();
        switch (columnName)
        {
            case "id": cache.Remove($"tx:{await replicationValue.Get<int>(stoppingToken)}"); break;
            case "credit":
            case "debit":
                var string? acc = await replicationValue.Get<string>(stoppingToken);
                acc = acc[..(acc.IndexOf('.') + 1)];
                cache.Remove("credit:" + acc);
                cache.Remove("debit:" + acc);
                break;
        }
    }
}
```

NoRedisDemo > C# Program.cs

```
    {
        // Always call SetReplicationStatus() or assign LastAppliedLsn and LastFlushedLsn individually
        // so that Npgsql can inform the server which WAL files can be removed/recycled.
        conn.SetReplicationStatus(message.WalEnd);
    }
}
}
```

```
}
return tx is { } Transaction? t ? Results.Ok(t) : Results.NotFound();
```

```
) =>
{
```

Postgres вместо Redis, продолжение

RedisDemo > C# Program.cs

```
178 app.MapPost("/prepare-cache", async ([FromQuery]int size,
179 [FromServices] AppDbContext ctx,
180 [FromServices] IDistributedCache cache,
181 CancellationToken ct
182 ) ⇒
183 {
184     var JsonDocument? json = JsonSerializer.SerializeToDocument(ctx.Transact
185     await cache.SetStringAsync("demo", json.RootElement.ToString()!, ct);
186 })
187 .WithName("PrepareCache");
188
189 app.MapPost("/test-cache", async (
190 [FromServices] IDistributedCache cache,
191 CancellationToken ct
192 ) ⇒
193 {
194     var string? s = await cache.GetStringAsync("demo", ct);
195     await cache.SetStringAsync("demo", s!, ct);
196 })
197 .WithName("TestCache");
198       You, 9 minutes ago • Uncommitted changes
199
200 app.Run();
```

NoRedisDemo > C# Program.cs

```
137 app.MapPost("/prepare-cache", async ([FromQuery]int size,
138 [FromServices] AppDbContext ctx,
139 CancellationToken ct
140 ) ⇒
141 {
142     var JsonDocument? json = JsonSerializer.SerializeToDocument(ctx.Transactions.Take(size).AsQueryable());
143     await ctx.Database.ExecuteSqlAsync($"insert into cache values({"demo"},{DateTime.UtcNow},{json})");
144 })
145 .WithName("PrepareCache");
146
147 app.MapPost("/test-cache", async (
148 [FromServices] AppDbContext ctx,
149 CancellationToken ct
150 ) ⇒
151 {
152     var IQueryable<CacheEntry>? q = ctx.Cache.Where(CacheEntry c ⇒ c.Key == "demo");
153     var JsonDocument? json = await q.Select(CacheEntry c ⇒ c.Contents).FirstAsync(ct);
154     await q.ExecuteUpdateAsync(UpdateSettersBuilder<CacheEntry> p ⇒ p.SetProperty(CacheEntry c ⇒ c.Contents, json), ct);
155 })
156 .WithName("TestCache");
157
158
159 app.Run();
```

Ограничения кэша в Postgres

- Масштабируемость
  - Только мастер
  - Запись больших объемов данных работает медленно
  - Использует shared buffers

- Остальные возможности Redis (если нет замены в Postgres)
  - Скрипты
  - Вероятностные структуры данных
  - Списки
  - TimeSeries
  - Векторы
  - ….

Заключение

- ~~«Using the right tool for the job»~~
- Используйте Postgres, пока не столкнулись с проблемами масштабирования
- Очереди
  - Logical replication – чтобы получили все
  - SELECT FOR UPDATE SKIP LOCKED – чтобы обработано было одним из получателей
- Кэш
  - Кэш памяти приложения + сброс через logical replication и notifications
  - MATERIALIZED VIEW
  - UNLOGGED таблицы
- Не забываем оптимизировать запросы

Код из слайдов https://github.com/gandjustas/pg-bootcamp2026-demo

Ваши вопросы?

PG BootCamp Russia 2026 Moscow

PGBootCamp.ru